Programming for Analytics

[Course work]

Vinthan Vinodini

Student ID: 23022705

Introduction

The goal of this coursework is to create predictive models and extract actionable insights by thoroughly analyzing and transforming a market campaign dataset. The dataset includes 1500 customers sociodemographic characteristics and product ownership data. The target variable is the categorization of customers into high and low value groups according to how frequently they use affinity cards.

The primary objectives of this coursework encompass data understanding, preparation, analysis, exploration, and mining. All these activities contribute to a comprehensive comprehension of the dataset and the creation of prediction models. We seek to extract meaningful insights, identify significant variables, manage incomplete or inaccurate data, and eventually construct a logistic regression machine learning model for predictive analysis through a series of python programming tasks and data manipulation techniques.

1. DATA UNDERSTANDING

The metadata table was generated using python code, leveraging the pandas library. In order to create the metadata table, the code reads the marketing campaign data from CSV file and extracts relevant information. The code loops through every column in the dataset, recording basic data for all columns and descriptive statistics for numeric columns. It populates the metadata table with column descriptions, data types, unique values and counts. For numeric columns, additional statistics such as minimum, maximum, mean and standard deviation are calculated. (Fig 1)

```python
metadata = pd.DataFrame()
metadata = pd.DataFrame(columns=['Descriptions', 'Mean'])
Descriptions ={
    'CUST_ID' : 'Customer ID',
    'CUST_GENDER': 'Gender of the customer',
    'AGE' : 'Age of the customer',
    'CUST_MARITAL_STATUS' : 'Marital status of the customer',
    'COUNTRY_NAME' : 'Country of the customer',
    'CUST_INCOME_LEVEL' : 'Income level of the customer',
    'EDUCATION' : 'Education level of the customer',
    'OCCUPATION' : 'Occupation of the customer',
    'HOUSEHOLD_SIZE' : 'Household size',
    'YRS_RESIDENCE' : 'Years of residence',
    'AFFINITY_CARD' : 'Affinity card usage',
    'BULK_PACK_DISKETTES' : 'Bulk pack diskettes purchase',
    'FLAT_PANEL_MONITOR' : 'Flat panel monitor purchase',
    'HOME_THEATER_PACKAGE' : 'Home theater package purchase',
    'BOOKKEEPING_APPLICATION' : 'Bookkeeping application purchase',
    'PRINTER_SUPPLIES' : 'Printer supplies purchase',
    'Y_BOX_GAMES' : 'Y-box games purchase',
    'OS_DOC_SET_KANJI' : 'Opearting system document set (kanji) purchase',
    'COMMENTS' : 'Additional comments'
}
metadata['Data Type'] = data.dtypes
metadata['Unique Values'] = data.nunique()
metadata['Count'] = data.count()

for column in data.columns:
  metadata.loc[column,'Descriptions'] = Descriptions.get(column,'')

  if pd.api.types.is_numeric_dtype(data[column]):
    metadata.loc[column, 'Min'] = data[column].min()
    metadata.loc[column, 'Max'] = data[column].max()
    metadata.loc[column, 'Mean'] = data[column].mean()
    metadata.loc[column, 'Std Deviation'] = data[column].std()


metadata
```

FIG 1: Code for meta data table

	Descriptions	Mean	Data Type	Unique Values	Count	Min	Max	Std Deviation
CUST_ID	Customer ID	102250.5	int64	1500	1500	101501.0	103000.0	433.157015
CUST_GENDER	Gender of the customer	NaN	object	2	1500	NaN	NaN	NaN
AGE	Age of the customer	38.892	int64	66	1500	17.0	90.0	13.636384
CUST_MARITAL_STATUS	Marital status of the customer	NaN	object	7	1500	NaN	NaN	NaN
COUNTRY_NAME	Country of the customer	NaN	object	19	1500	NaN	NaN	NaN
CUST_INCOME_LEVEL	Income level of the customer	NaN	object	12	1500	NaN	NaN	NaN
EDUCATION	Education level of the customer	NaN	object	16	1500	NaN	NaN	NaN
OCCUPATION	Occupation of the customer	NaN	object	15	1500	NaN	NaN	NaN
HOUSEHOLD_SIZE	Household size	NaN	object	6	1500	NaN	NaN	NaN
YRS_RESIDENCE	Years of residence	4.088667	int64	15	1500	0.0	14.0	1.920919
AFFINITY_CARD	Affinity card usage	0.253333	int64	2	1500	0.0	1.0	0.435065
BULK_PACK_DISKETTES	Bulk pack diskettes purchase	0.628	int64	2	1500	0.0	1.0	0.483500
FLAT_PANEL_MONITOR	Flat panel monitor purchase	0.582	int64	2	1500	0.0	1.0	0.493395
HOME_THEATER_PACKAGE	Home theater package purchase	0.575333	int64	2	1500	0.0	1.0	0.494457
BOOKKEEPING_APPLICATION	Bookkeeping application purchase	0.880667	int64	2	1500	0.0	1.0	0.324288
PRINTER_SUPPLIES	Printer supplies purchase	1.0	int64	1	1500	1.0	1.0	0.000000
Y_BOX_GAMES	Y-box games purchase	0.286667	int64	2	1500	0.0	1.0	0.452355
OS_DOC_SET_KANJI	Opearting system document set (kanji) purchase	0.002	int64	2	1500	0.0	1.0	0.044692
COMMENTS	Additional comments	NaN	object	44	1427	NaN	NaN	NaN

FIG 2: Meta data table

In Fig 2, the meta data table shows that there are 1500 rows in the dataset. There are 19 features in the dataset which I will use in my data analysis as specified in the requirement document. There are no missing values in the dataset apart from the column comment, which includes 73 missing values. 11 of the 19 columns in the dataset are of the integer type and the remaining 8 are the object type.

```python
# Define a function to generate metadata for each column
def generate_metadata(column):
    metadata = {}
    metadata['Attribute Name'] = column.name
    metadata['Description'] = "Description of the attribute"  # Add descriptions as per your knowledge

    if column.dtype == 'object':  # Check if the column is nominal
        metadata['Mode'] = column.mode()[0]
        plt.figure(figsize=(8, 6))
        column.value_counts().plot(kind='bar')
        plt.title(f'{column.name} Distribution')
        plt.xlabel(column.name)
        plt.ylabel('Frequency')
        plt.show()
    else:  # Numeric data
        try:
            column_numeric = pd.to_numeric(column, errors='coerce')  # Convert to numeric, coerce errors to NaN
            column_numeric = column_numeric.dropna()  # Drop NaN values
            metadata['Maximum'] = column_numeric.max()
            metadata['Minimum'] = column_numeric.min()
            metadata['Mean'] = column_numeric.mean()
            metadata['Std. Deviation'] = column_numeric.std()

            plt.figure(figsize=(8, 6))
            plt.hist(column_numeric, bins=10, color='skyblue', edgecolor='black')
            plt.title(f'{column.name} Distribution')
            plt.xlabel(column.name)
            plt.ylabel('Frequency')
            plt.show()
        except ValueError:
            print(f"Warning: {column.name} contains non-numeric values and will be excluded from numeric metadata.")

    return metadata

# Generate metadata for each column
metadata_list = []
for column in df.columns:
    metadata_list.append(generate_metadata(df[column]))

# Convert metadata list to DataFrame for easy viewing
metadata_df = pd.DataFrame(metadata_list)
```

FIG 3: Code for histogram and bar chart

A histogram is a graph that shows how numerical data is distributed. It is made up of a sequence of adjacent rectangles called bins, each of which stands for a certain range of values. The form, central tendency and dispersion of the distribution of numerical variables can all be seen via histograms. They can highlight significant features including multimodality, skewness, and symmetry. Nominal data consists of categorical variables that represent discrete categories or labels without any inherent order. It represents the central tendency of the distribution for categorical data.

Essential tools for exploratory data analysis are bar charts and histograms, which give analysis insights into the frequencies and distributions of numerical and nominal data respectively. They make it easier to find trends, patterns, and outliers in the dataset, which helps with subsequent analysis and decision making.

In FIG 2 shows the code for histogram and bar chart for the entire dataset. To create metadata, the function generate_ metadata(column) loops through each column in the dataset. It determines the mode (most frequent value) using nominal (categorical) data and produces a bar chart to show the frequency distribution. When dealing with numerical data, it computes statistics like the mean, standard deviation, maximum and minimum and produces a histogram to show the distribution.

In the results, histograms are generated for the 'AGE' and 'Years of residence' columns, while bar charts are produced for all other variables.

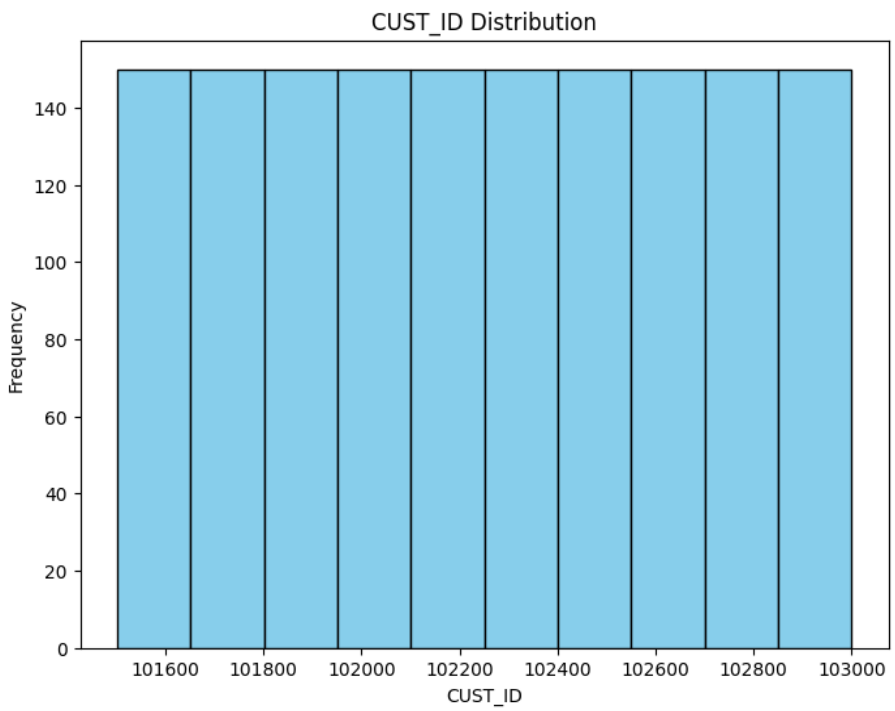


FIG 4: Customet ID distribution

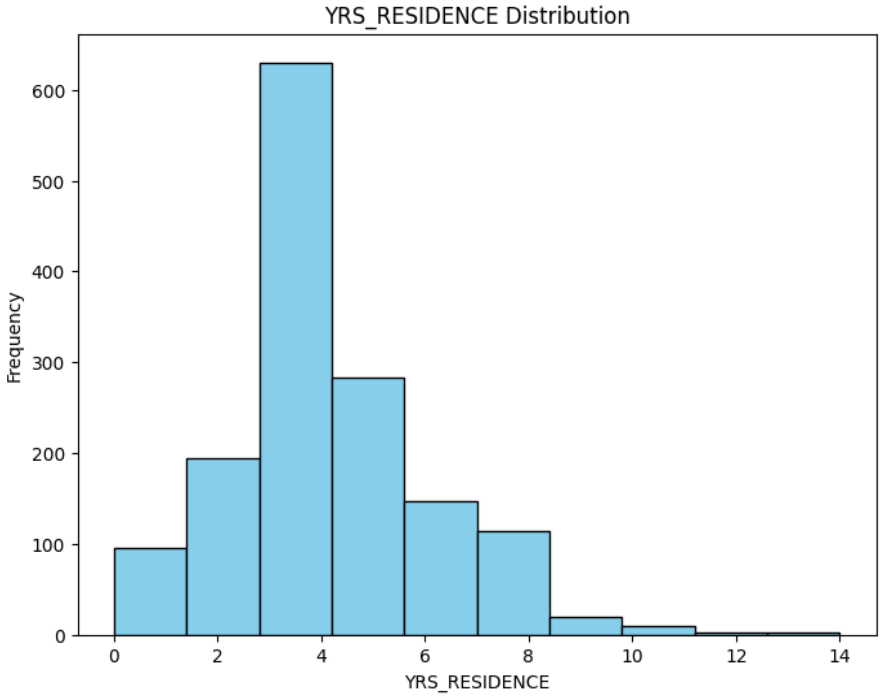


FIG 5: Histogram for years of residence

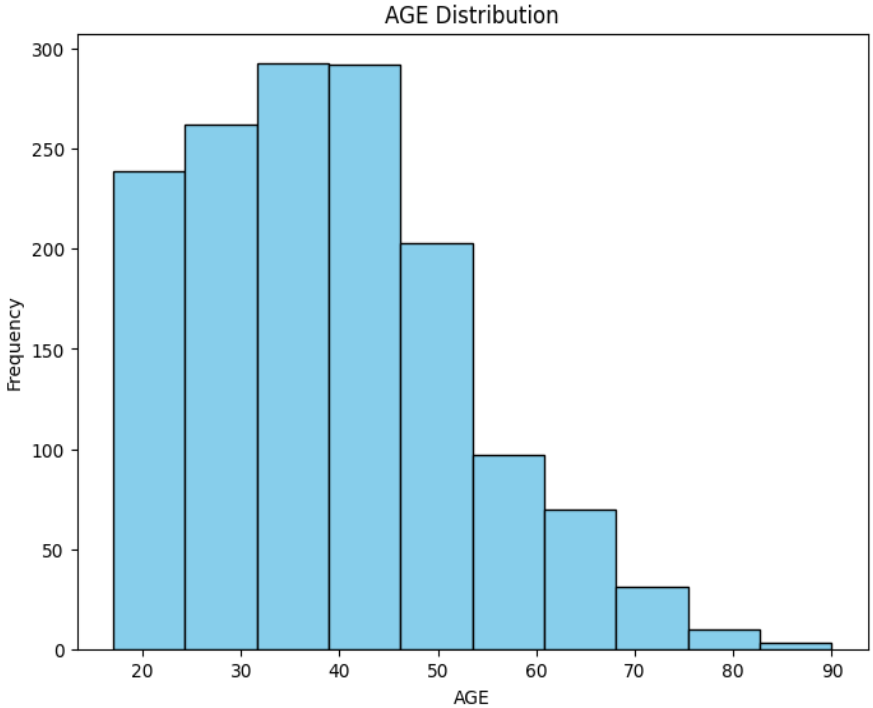


FIG 6: Histogram for Age

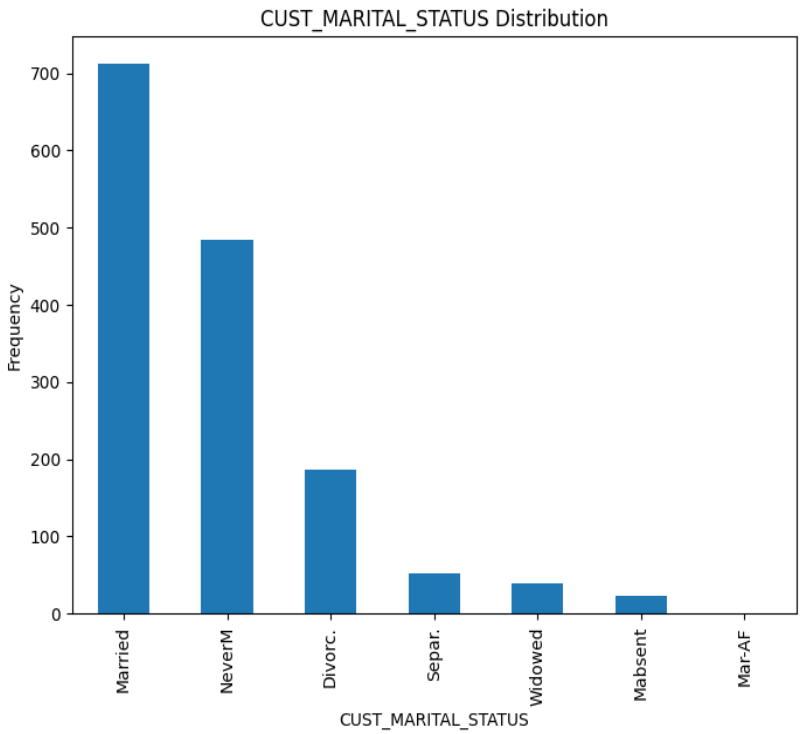


FIG 7: Bar chart for Customer marital status

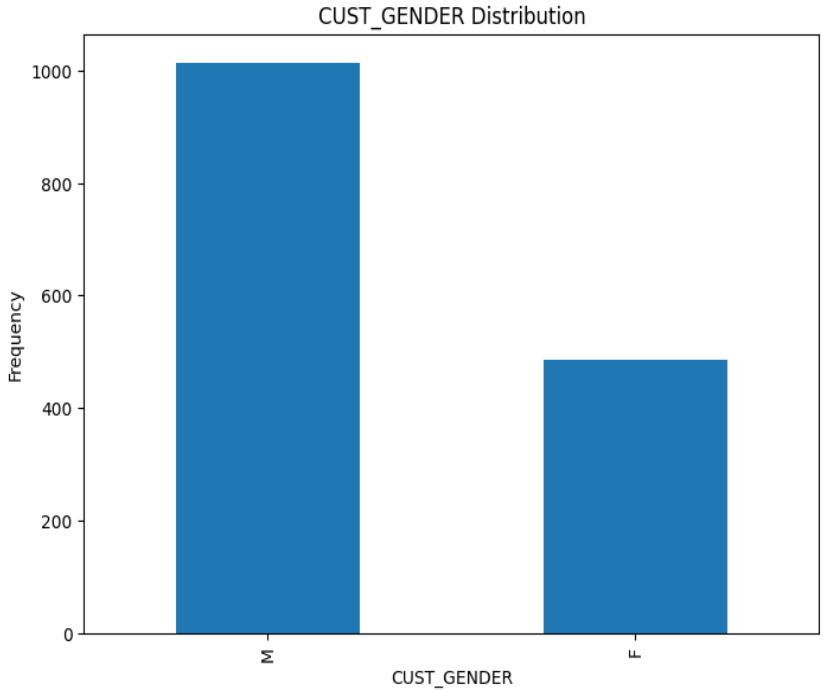


FIG 8: Bar chart for customer gender

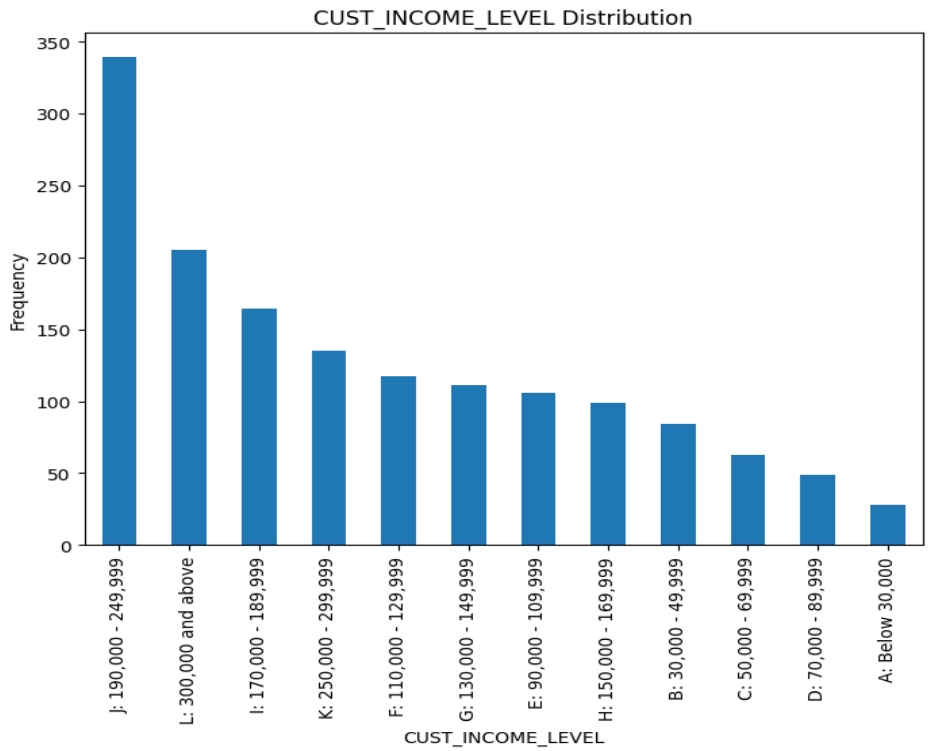


FIG 9 : Bar chart for customer income level

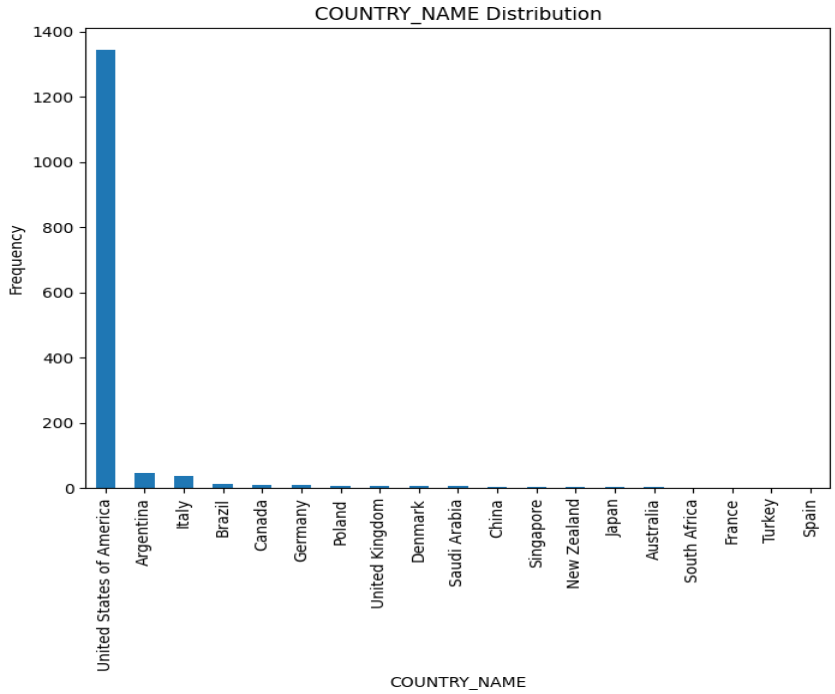


FIG 10: Bar chart for country name

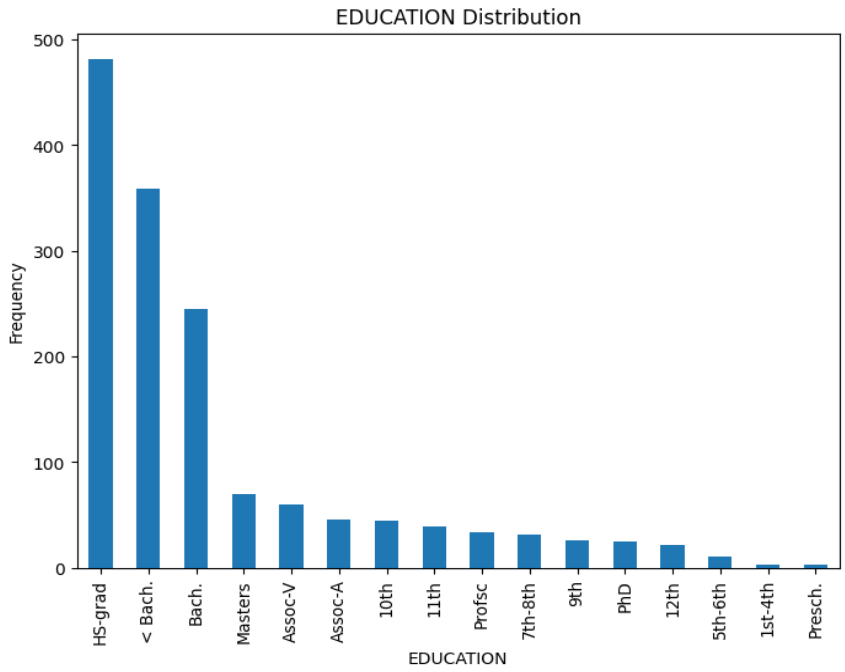


FIG 11: Bar chart for Education

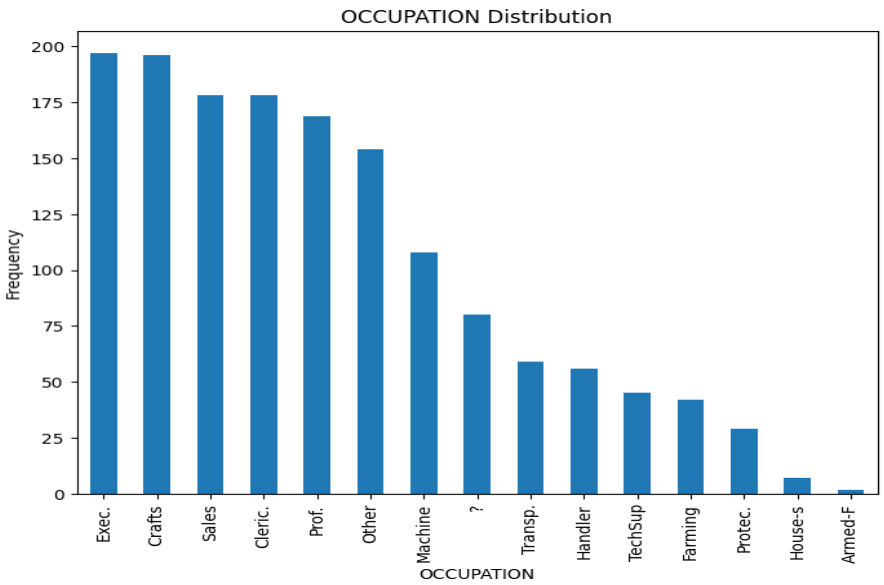


FIG 12: Bar chart for Occupation

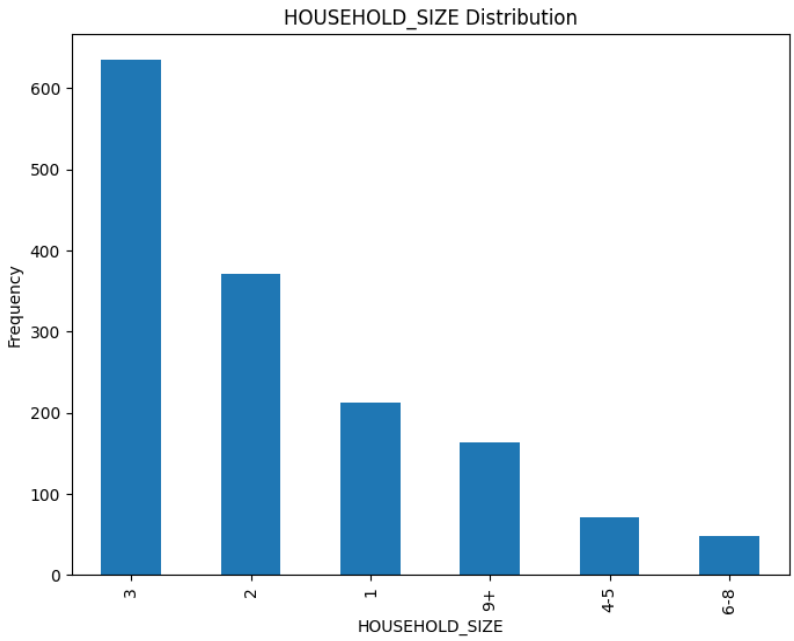


FIG 13: Bar chart for Household size

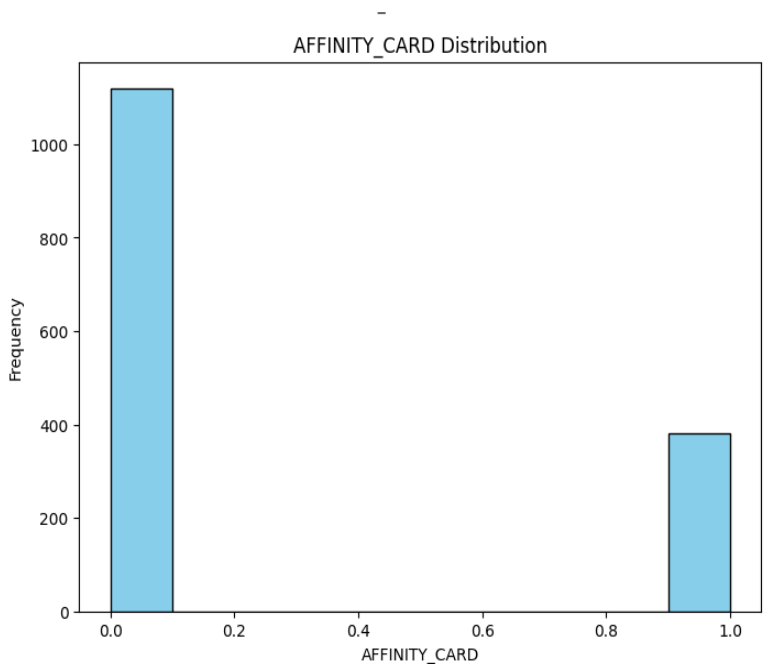


FIG 14: Bar chart for Affinity card

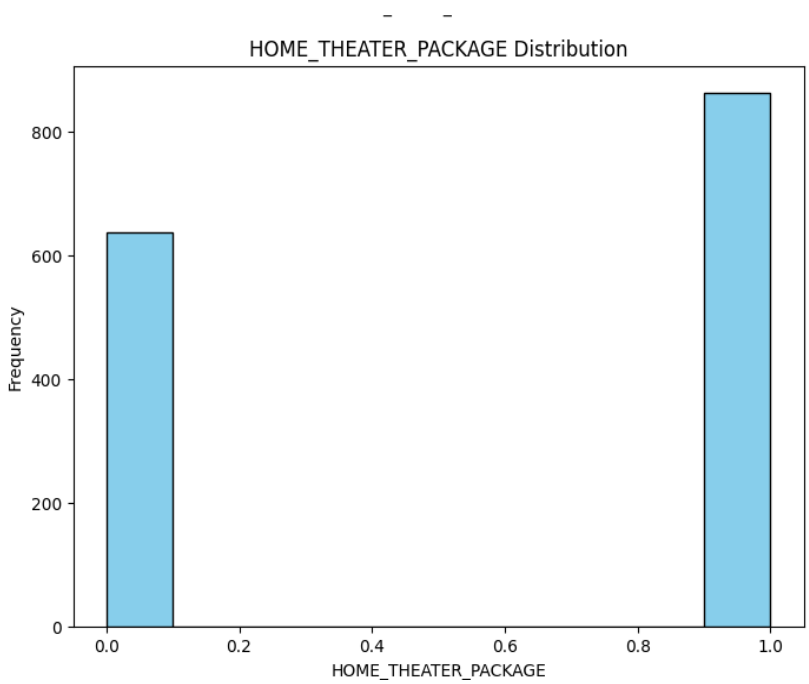


FIG 15: Bar chart for Home theater package

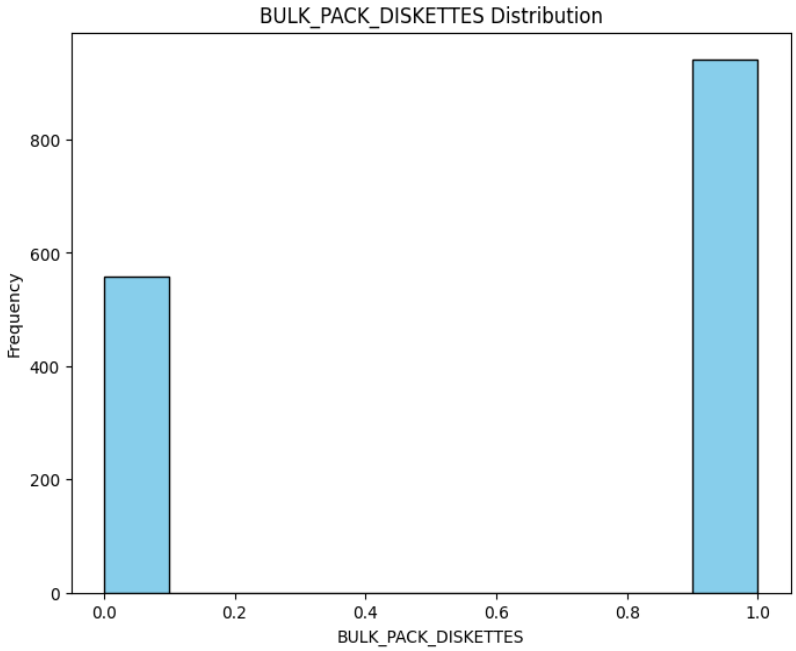


FIG 16: Bar chart for Bulk pack diskettes

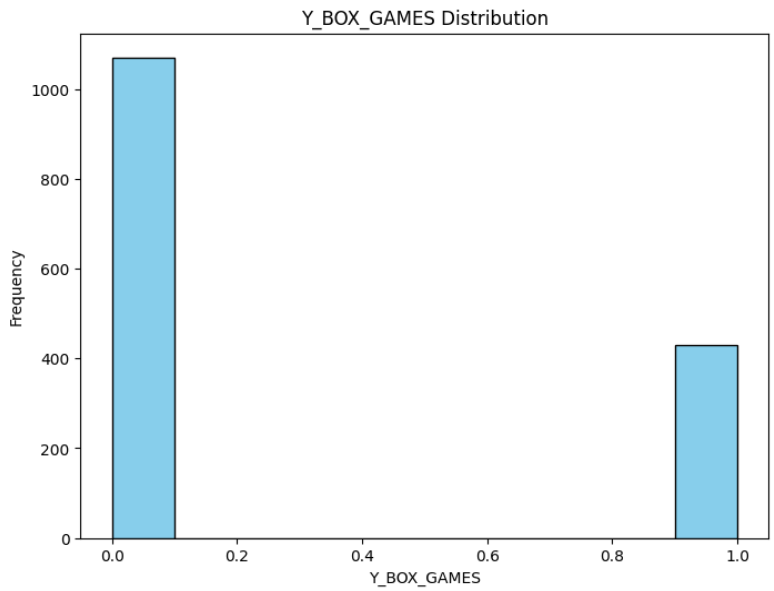


FIG17: Bar chart for Y box games

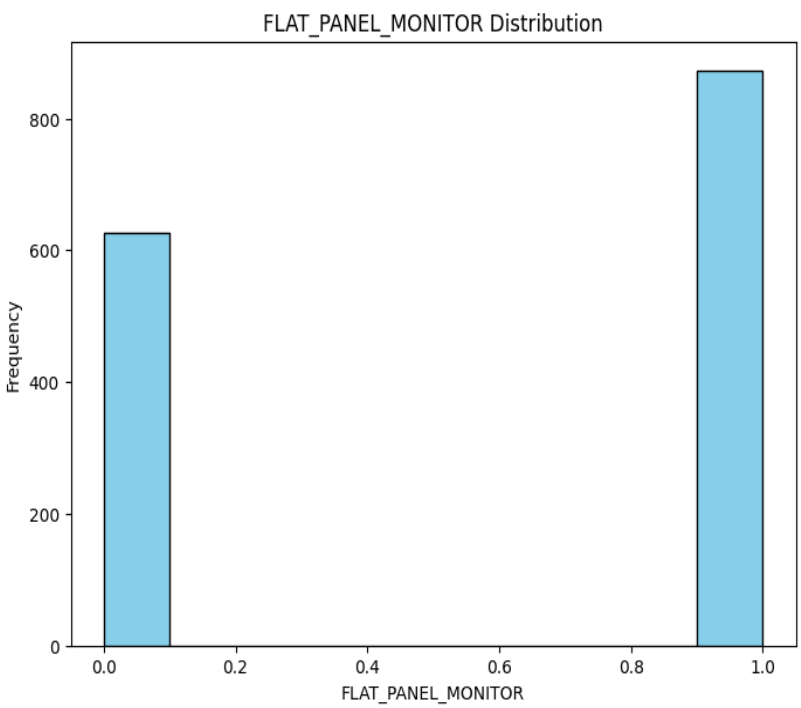


FIG 18: Bar chart for flat panel monitor

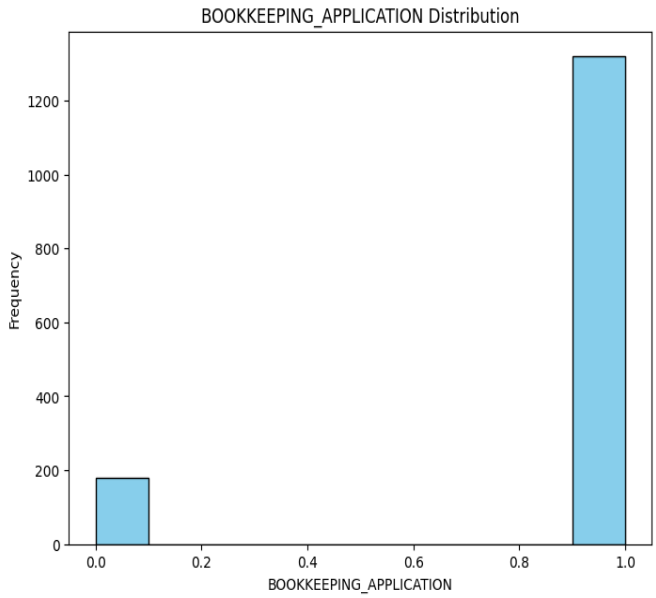


FIG 19: Bar chart for Bookkeeping Application

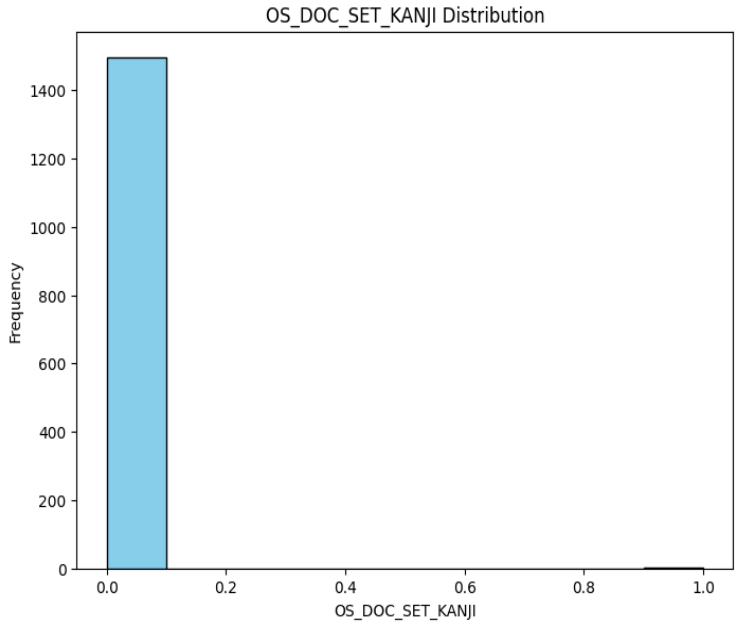


FIG 21: Bar chart for Os doc set kanji

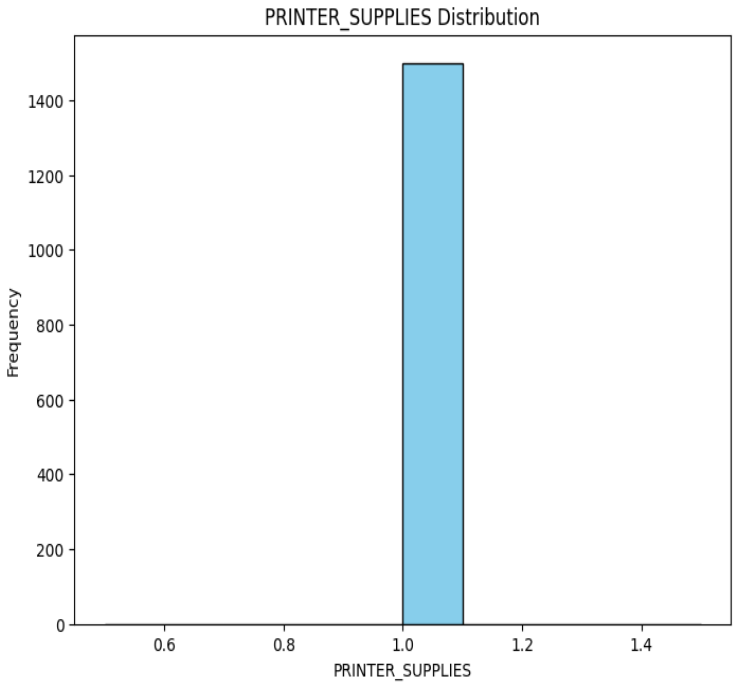


FIG22: Bar chart for Printer supplies

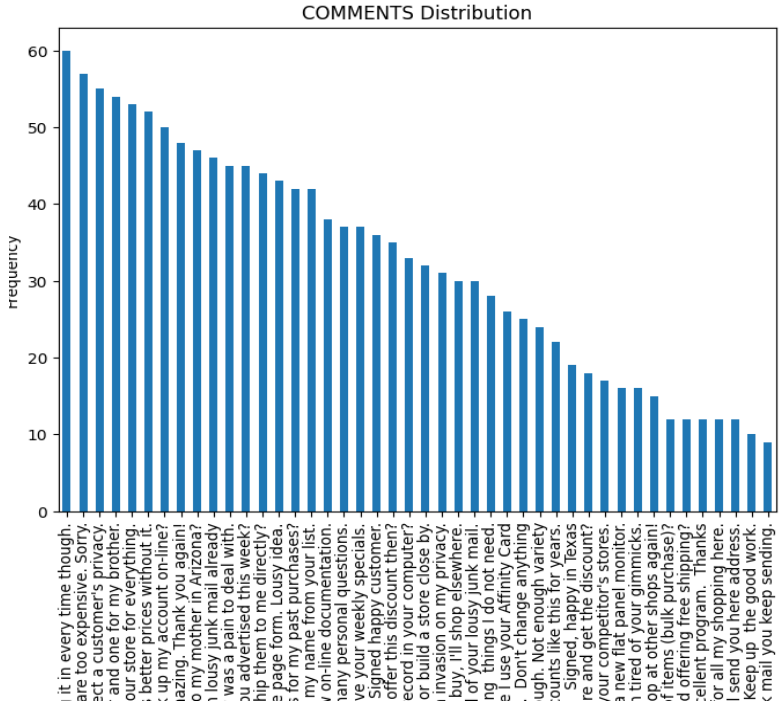


FIG 23: Bar chart for comments

```python
# Define the list of column names
column_names = ['CUST_GENDER', 'CUST_MARITAL_STATUS', 'COUNTRY_NAME', 'CUST_INCOME_LEVEL', 'EDUCATION', 'OCCUPATION', 'AFFINITY_CARD', 'BULK_PACK_DISKETTES', 'FLAT_PANEL_MONITOR', 'HOME_THEATER_PACKA

# Calculate mode for each nominal attribute
mode_values = data[column_names].mode().iloc[0]
mode_values.index.name = 'Attribute'
mode_values.name = 'Mode'

print("Mode for each nominal attribute:")
print(mode_values)
```

FIG 24: Code for Mode

```
Mode for each nominal attribute:
Attribute
CUST_GENDER                                     M
CUST_MARITAL_STATUS                       Married
COUNTRY_NAME               United States of America
CUST_INCOME_LEVEL              J: 190,000 - 249,999
EDUCATION                                 HS-grad
OCCUPATION                                  Exec.
AFFINITY_CARD                                   0
BULK_PACK_DISKETTES                             1
FLAT_PANEL_MONITOR                              1
HOME_THEATER_PACKAGE                            1
BOOKKEEPING_APPLICATION                         1
PRINTER_SUPPLIES                                1
Y_BOX_GAMES                                     0
OS_DOC_SET_KANJI                                0
Name: Mode, dtype: object
```

FIG 25: Mode for each attribute

In statistics, the mode is a measure of central tendency that represents the most frequently occurring value in a dataset. It is particularly useful for categorical or nominal data, where values are distinct categories rather than numerical quantities.

In Fig 24 and 25 show the code and results of the mode. The results show the most frequent values of each attribute.

```python
# Define expected patterns or ranges for each attribute (if applicable)
expected_patterns_or_ranges = {
    'CUST_ID': None,
    'AGE': (17, 90),
    'GENDER': ['Male', 'Female'],
    'INCOME': (0, None),
    'EDUCATION': ['High School', 'Bachelor', 'Master', 'PhD','Masters', 'Bach.', 'HS-grad','< Bach.', 'Profsc', '11th', '< Bach.', '5th-6th','Assoc-A', 'Assoc-V', '7th-8th', '7th-8tl
    'OCCUPATION': ['Armed-F', 'Cleric.', 'Crafts', 'Exec.', 'Farming', 'Handler', 'House-s', 'Machine', 'Other', 'Prof.', 'Protec.', 'Sales', 'TechSup', 'Transp.'],
    'HOUSEHOLD_SIZE': ('1', '2', '3', '4-5', '6-8', '9+'),
    'YRS_RESIDENCE': list(range(15)),
    'AFFINITY_CARD': [0, 1],
    'BULK_PACK_DISKETTES': [0, 1],
    'FLAT_PANEL_MONITOR': [0, 1],
    'HOME_THEATER_PACKAGE': [0, 1],
    'BOOKKEEPING_APPLICATION': [0, 1],
    'PRINTER_SUPPLIES': [1],
    'Y_BOX_GAMES': [0, 1],
    'OS_DOC_SET_KANJI': [0, 1],
    'COMMENTS': None
}

# Initialize dictionaries to store missing and error data counts
missing_data_counts = {}
error_data_counts = {}
error_data_types = {}

# Iterate over each attribute to count missing and error data
for attribute in data.columns:
    # Count missing data types: null, blank, unknown
    missing_data_count = data[attribute].isnull().sum() + data[attribute].eq('').sum()
    missing_data_counts[attribute] = missing_data_count

    # Initialize error data count and types for the attribute
    error_data_count = 0
    error_data_types[attribute] = []
```

```python
    if attribute in expected_patterns_or_ranges:
        expected_values = expected_patterns_or_ranges[attribute]
        if expected_values is not None:
            if isinstance(expected_values, tuple) and len(expected_values) == 2:  # Check for numerical range
                min_val, max_val = expected_values
                for value in data[attribute]:
                    if not (min_val <= value <= max_val):
                        error_data_count += 1
                        error_data_types[attribute].append(value)
            elif isinstance(expected_values, list):  # Check for categorical values
                for value in data[attribute]:
                    if value not in expected_values:
                        error_data_count += 1
                        error_data_types[attribute].append(value)

    # Store error data count for the attribute
    error_data_counts[attribute] = error_data_count

# Print counts for missing and error data in each attribute
for attribute in data.columns:
    print(f"Attribute: {attribute}")
    print(f"Missing Data Count: {missing_data_counts.get(attribute, 0)}")
    print(f"Error Data Count: {error_data_counts.get(attribute, 0)}")
    print(f"Error Data Types: {error_data_types.get(attribute, [])}")
    print()
```

FIG 26: Code for missing and error values

It is important to investigate whether any of the dataset's attributes have missing and error data, and have to provide suggested solutions for resolving these discrepancies. By understanding and effectively managing these issues, we aim to ensure the integrity of our analyses and conclusions.

```
Attribute: CUST_GENDER
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: AGE
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: CUST_MARITAL_STATUS
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: COUNTRY_NAME
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: CUST_INCOME_LEVEL
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: EDUCATION
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: OCCUPATION
Missing Data Count: 0
Error Data Count: 80
Error Data Types: ['?', '?', '?', '?', '?', '?', '?', '?', '?', '?', '?', '?', '?', '?', '?', '
```

FIG 27: Results for Missing and error values (1)

In FIG 27 and Fig 28, shows the findings related to missing and error values within the entirely of the dataset. My research shows that the 'comments' column has 73 missing values, and the 'occupation' column has 80 occurrences of error data indicated by the question mark '?'. It is observed that all other columns in the dataset, excluding 'Occupation and 'Comments' are free of missing and error values.

```
Attribute: HOUSEHOLD_SIZE
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: YRS_RESIDENCE
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: AFFINITY_CARD
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: BULK_PACK_DISKETTES
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: FLAT_PANEL_MONITOR
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: HOME_THEATER_PACKAGE
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: BOOKKEEPING_APPLICATION
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: PRINTER_SUPPLIES
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: Y_BOX_GAMES
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: OS_DOC_SET_KANJI
Missing Data Count: 0
Error Data Count: 0
Error Data Types: []

Attribute: COMMENTS
Missing Data Count: 73
Error Data Count: 0
Error Data Types: []
```

FIG 28: Results for missing and error values (11)

2. DATA PREPARATION

In this section will cover data cleaning and reducing variable in a python environment. The main goal is to reduce the number of variables in the dataset that have no influence on the target variable. Furthermore, data protocols will be employed to address error and missing values, with a particular emphasis on records that have equal to or less than 5% of these instances.

```
[ ] # Remove non-numeric columns and the target variable (AFFINITY_CARD) from the dataset
    numeric_data = data.select_dtypes(include=['int', 'float'])

    # Calculate correlation between each attribute and the target variable
    correlation = numeric_data.corrwith(data['AFFINITY_CARD']).drop('AFFINITY_CARD', errors='ignore')

    # Identify variables with zero correlation (or close to zero)
    zero_influence_variables = correlation[correlation.abs() < 0.05].index.tolist()

    # Remove variables with zero influence from the dataset
    reduced_data = data.drop(columns=zero_influence_variables)

    # Print the reduced dataset
    print("Zero influence variable")
    print(zero_influence_variables)

    Zero influence variable
    ['CUST_ID', 'BULK_PACK_DISKETTES', 'FLAT_PANEL_MONITOR', 'OS_DOC_SET_KANJI']
```

FIG 29: Zero influence variable

In figure 29 shows the code aimed to find variables with zero influence on the target variable. For this analysis, I set a correlation threshold below 0.05, which denotes a minimal association with the target. Following the correlation study, I discovered that the following four variables such as CUST_ID, BULK_PACK_DISKETTES, FLAT_PANEL_MONITOR, and OS_DOS_SET_KANJI show negligible influence on the target variables.

These variables are considered negligible for predicting the results of the target variable because their correlations are below the defined threshold. Consequently, to improve model efficiency and streamline the dataset, they are taken into consideration for elimination in further data processing.

```python
# Assuming 'AFFINITY_CARD' is the target variable
target_variable = 'AFFINITY_CARD'

# Function to identify variables with zero influence on the target variable
def identify_zero_influence_variables(data, target_variable):
    influence_threshold = 0.05  # Adjust threshold as needed
    zero_influence_variables = []
    for column in data.columns:
        if column != target_variable:
            if pd.api.types.is_numeric_dtype(data[column]):  # Check if the column is numeric
                correlation = data[[column, target_variable]].corr().iloc[0, 1]
                if abs(correlation) < influence_threshold:
                    zero_influence_variables.append(column)
            else:
                print(f"Skipping non-numeric column '{column}'")
    return zero_influence_variables

# Function to reduce variables based on zero influence and COMMENTS
def reduce_variables(data, target_variable):
    # Identify variables with zero influence on the target variable
    zero_influence_vars = identify_zero_influence_variables(data, target_variable)

    # Remove 'COMMENTS' variable regardless of its correlation with the target variable
    if 'COMMENTS' in data.columns:
        zero_influence_vars.append('COMMENTS')

    # Drop identified variables
    reduced_data = data.drop(zero_influence_vars, axis=1)
    print("Zero-influeced removed columns:",zero_influence_vars)

    return reduced_data

# Reduce variables based on the defined criteria
reduced_df = reduce_variables(data, target_variable)

# Print the reduced DataFrame
reduced_df
```

FIG 30: Code for reduced Zero influence variable

In Fig 30, I outline the python code demonstrating the process of reducing variables that exhibit zero influence on the dataset. After identifying variables with negligible impact on the target variable, as shown in Fig 29, I proceed to remove these variables from the dataset.

On the other hand, it has been found that it requires the employment of text mining methods to handle the 73 missing values and it is considered as the zero-influence variable with target variable (results in FIG 31). As a result, it has been decided to completely remove the comment column from the data set.

To simplify the dataset, the code systematically removes the zero-influence variables with comments. By reducing these variables to enhance the model's efficiency and computational performance, as it focuses only on the most relevant features for predicting the target variable

```
Skipping non-numeric column 'CUST_GENDER'
Skipping non-numeric column 'CUST_MARITAL_STATUS'
Skipping non-numeric column 'COUNTRY_NAME'
Skipping non-numeric column 'CUST_INCOME_LEVEL'
Skipping non-numeric column 'EDUCATION'
Skipping non-numeric column 'OCCUPATION'
Skipping non-numeric column 'HOUSEHOLD_SIZE'
Skipping non-numeric column 'COMMENTS'
Zero-influeced removed columns: ['CUST_ID', 'BULK_PACK_DISKETTES', 'FLAT_PANEL_MONITOR', 'OS_DOC_SET_KANJI', 'COMMENTS']
```

	CUST_GENDER	AGE	CUST_MARITAL_STATUS	COUNTRY_NAME	CUST_INCOME_LEVEL	EDUCATION	OCCUPATION	HOUSEHOLD_SIZE	YRS_RESIDENCE	AFFINITY_CARD	HOME_THEATER_PACKAGE	BOOKKEEPING_A
0	F	41	NeverM	United States of America	J: 190,000 - 249,999	Masters	Prof.	2	4	0	1	
1	M	27	NeverM	United States of America	I: 170,000 - 189,999	Bach.	Sales	2	3	0	0	
2	F	20	NeverM	United States of America	H: 150,000 - 169,999	HS-grad	Cleric.	2	2	0	0	
3	M	45	Married	United States of America	B: 30,000 - 49,999	Bach.	Exec.	3	5	1	1	
4	M	34	NeverM	United States of America	K: 250,000 - 299,999	Masters	Sales	9+	5	1	0	
...	...	...	...	...	...	...	...	...	...	...	...	
1495	M	17	NeverM	United States of America	C: 50,000 - 69,999	10th	Other	1	1	0	0	
1496	M	41	Married	Spain	L: 300,000 and above	Bach.	Exec.	3	4	0	1	
1497	M	53	Married	United States of America	J: 190,000 - 249,999	HS-grad	Exec.	3	8	1	1	
1498	M	55	Married	United States of America	C: 50,000 - 69,999	HS-grad	Cleric.	3	7	1	1	
1499	F	40	Divorc.	United States of America	E: 90,000 - 109,999	HS-grad	Cleric.	2	3	0	1	

1500 rows × 14 columns

FIG 31: Results of rest columns after dropping

In Fig 31, the outcome of dropping the zero influence variables is depicted, accompanied by explanatory comments. After removing the identified zero influence variables with comments from the dataset, there are **14 columns** remaining out of the initial 19 columns.

```python
# Identify rows with '?' in the 'occupation' column
rows_with_question_mark = reduced_df[reduced_df['OCCUPATION'] == '?']

# Print the rows with '?' in the 'occupation' column
print("Rows with '?' in the 'OCCUPATION' column:")
print(rows_with_question_mark)

# Remove rows with '?' in the 'occupoccation' column
cleaned_df = reduced_df[reduced_df['OCCUPATION'] != '?']

# Print the number of rows after handling '?' values
print("Number of rows after handling '?' values:", len(cleaned_df))
```

FIG 32: Code for dropping error data in occupation column

I decided to evaluate the proportion of error data relative to the whole dataset to address the problem with occupation column. It has been determined that is it suitable to eliminate the error data because the error count in the occupation column represents around 5%. In Fig 32 shows the code for dropping error data in occupation.

```
Rows with '?' in the 'OCCUPATION' column:
[ ]      CUST_GENDER  AGE CUST_MARITAL_STATUS              COUNTRY_NAME  \
     38            F   69             Widowed                     Italy
     39            F   35              NeverM  United States of America
     55            F   64             Widowed  United States of America
     57            F   27              Separ.  United States of America
     74            F   28             Married                 Australia
     ...         ...  ...                ...                       ...
     1368          M   61             Married  United States of America
     1388          F   33             Divorc.  United States of America
     1446          M   23              NeverM  United States of America
     1456          F   54              NeverM  United States of America
     1492          M   69             Married  United States of America

              CUST_INCOME_LEVEL EDUCATION OCCUPATION HOUSEHOLD_SIZE  YRS_RESIDENCE  \
     38    I: 170,000 - 189,999   Assoc-A          ?              2              2
     39    L: 300,000 and above   < Bach.          ?             9+              4
     55     E: 90,000 - 109,999   < Bach.          ?             9+              2
     57    J: 190,000 - 249,999       9th          ?             9+              3
     74    L: 300,000 and above   HS-grad          ?            4-5              1
     ...                    ...       ...        ...            ...            ...
     1368  J: 190,000 - 249,999   HS-grad          ?              3              6
     1388  L: 300,000 and above   < Bach.          ?              2              3
     1446  L: 300,000 and above   < Bach.          ?              2              2
     1456  H: 150,000 - 169,999   Masters          ?              2              5
     1492  F: 110,000 - 129,999       9th          ?              3              3

           AFFINITY_CARD  HOME_THEATER_PACKAGE  BOOKKEEPING_APPLICATION  \
     38                0                     1                        1
     39                0                     1                        1
     55                0                     1                        1
     57                0                     0                        0
     74                0                     0                        1
     ...             ...                   ...                      ...
     1368              1                     1                        1
     1388              0                     0                        1
     1446              0                     0                        1
     1456              0                     1                        1
     1492              0                     1                        0

           PRINTER_SUPPLIES  Y_BOX_GAMES
     38                   1            0
     39                   1            0
     55                   1            0
     57                   1            1
     74                   1            1
     ...                ...          ...
     1368                 1            0
     1388                 1            0
     1446                 1            1
     1456                 1            0
     1492                 1            0

     [80 rows x 14 columns]
     Number of rows after handling '?' values: 1420
```

FIG 33: Results of dropping error rows and zero influence column

Fig 33 shows the finalized dataset intended for analysis. Within the occupation column 80 rows have been dropped due to error data.

Finally, **14 columns and 1420 rows** of data are deemed relevant for subsequent analysis and will be utilized in further modeling and exploration tasks.

3. DATA TRANSFORMATION

The process of transformation data from one format to a more appropriate one is known as data transformation. The specification includes transformation rules that indicate how to turn variables into meaningful data for machine learning. The process starts from converting variables into ordinal and binary numbers which also changes the data type of the variable.

CUST_GENDER

Fig 34 demonstrates the process of transforming the CUST_GENDER column into a binary representation, where 'F' is encoded as 0 and 'M' is encoded as 1.

```
[ ] cleaned_df.loc[:,'CUST_GENDER'] = cleaned_df['CUST_GENDER'].map({'F':0,'M':1})
```

```
[ ] cleaned_df.head()
```

FIG 34: Mapping gender into Binary

	CUST_GENDER	AGE	CUST_MARITAL_STATUS	COUNTRY_NAME	CUST_INCOME_LEVEL	EDUCATION	OCCUPATION	HOUSEHOLD_SIZE	YRS_RESIDENCE	AFFINITY_CARD	HOME_THEATER_PACKAGE	BOOKKEEPING_APPLI
0	0	41	NeverM	United States of America	J: 190,000 - 249,999	Masters	Prof.	2	4	0	1	
1	1	27	NeverM	United States of America	I: 170,000 - 189,999	Bach.	Sales	2	3	0	0	
2	0	20	NeverM	United States of America	H: 150,000 - 169,999	HS-grad	Cleric.	2	2	0	0	
3	1	45	Married	United States of America	B: 30,000 - 49,999	Bach.	Exec.	3	5	1	1	
4	1	34	NeverM	United States of America	K: 250,000 - 299,999	Masters	Sales	9+	5	1	0	

FIG 35: Result of gender transformation

Fig 35 shows the outcome of gender transformation, where CUST_GENDER column has been encoded into binary values.

COUNTRY_NAME

Fig 36 shows the code snippet transforms the COUNTRY_NAME column in the data frame by mapping each unique country name to an ordinal number, sorting them based on their occurrence frequency.

```python
# Get unique country names and sort them in ascending order of occurrence
country_order = cleaned_df['COUNTRY_NAME'].value_counts().sort_index().index

# Map country names to ordinal numbers
country_mapping = {country: i for i, country in enumerate(country_order)}

# Replace country names with ordinal numbers using .loc accessor
cleaned_df.loc[:, 'COUNTRY_NAME'] = cleaned_df['COUNTRY_NAME'].map(country_mapping)

# Display the transformed DataFrame
print(cleaned_df.head())

# Create a dictionary to map ordinal numbers back to country names
ordinal_to_country = {i: country for i, country in enumerate(country_order)}

# Display the ordinal number and corresponding country name
for ordinal, country in ordinal_to_country.items():
    print(f"Ordinal Number: {ordinal}, Country Name: {country}")
```

FIG 36: Country name into ordinal numbers

```
[ ]   CUST_GENDER  AGE CUST_MARITAL_STATUS COUNTRY_NAME     CUST_INCOME_LEVEL  \
    0           0   41              NeverM           18  J: 190,000 - 249,999
    1           1   27              NeverM           18  I: 170,000 - 189,999
    2           0   20              NeverM           18  H: 150,000 - 169,999
    3           1   45             Married           18    B: 30,000 - 49,999
    4           1   34              NeverM           18  K: 250,000 - 299,999

      EDUCATION OCCUPATION HOUSEHOLD_SIZE  YRS_RESIDENCE  AFFINITY_CARD  \
    0   Masters      Prof.              2              4              0
    1     Bach.      Sales              2              3              0
    2   HS-grad    Cleric.              2              2              0
    3     Bach.      Exec.              3              5              1
    4   Masters      Sales             9+              5              1

       HOME_THEATER_PACKAGE  BOOKKEEPING_APPLICATION  PRINTER_SUPPLIES  \
    0                     1                        1                 1
    1                     0                        1                 1
    2                     0                        1                 1
    3                     1                        1                 1
    4                     0                        1                 1

       Y_BOX_GAMES
    0            0
    1            1
    2            1
    3            0
    4            0
    Ordinal Number: 0, Country Name: Argentina
    Ordinal Number: 1, Country Name: Australia
    Ordinal Number: 2, Country Name: Brazil
    Ordinal Number: 3, Country Name: Canada
    Ordinal Number: 4, Country Name: China
    Ordinal Number: 5, Country Name: Denmark
    Ordinal Number: 6, Country Name: France
    Ordinal Number: 7, Country Name: Germany
    Ordinal Number: 8, Country Name: Italy
    Ordinal Number: 9, Country Name: Japan
    Ordinal Number: 10, Country Name: New Zealand
    Ordinal Number: 11, Country Name: Poland
    Ordinal Number: 12, Country Name: Saudi Arabia
    Ordinal Number: 13, Country Name: Singapore
    Ordinal Number: 14, Country Name: South Africa
    Ordinal Number: 15, Country Name: Spain
    Ordinal Number: 16, Country Name: Turkey
    Ordinal Number: 17, Country Name: United Kingdom
    Ordinal Number: 18, Country Name: United States of America
```

FIG 37: Results of country name transformation

CUST_INCOME_LEVEL

Fig 38 shows the code of a mapping dictionary, ‘income mapping’, which categorizes income levels into ordinal categories representing low income, middle income and high-income brackets based on the specified income ranges.

```
income_map = {'low': 1, 'middle': 2, 'high': 3}
# Define the mapping of income levels to ordinal categories
income_mapping = {
   'A: Below 30,000': 1,          # Low income
    'B: 30,000 - 49,999': 1,        # Low income
    'C: 50,000 - 69,999': 1,        # Low income
    'D: 70,000 - 89,999': 1,        # Low income
    'E: 90,000 - 109,999': 2,       # Middle income
    'F: 110,000 - 129,999': 2,      # Middle income
    'G: 130,000 - 149,999': 2,      # Middle income
   'H: 150,000 - 169,999': 3,      # High income
    'I: 170,000 - 189,999': 3,      # High income
    'J: 190,000 - 249,999': 3,      # High income
    'K: 250,000 - 299,999': 3,      # High income
    'L: 300,000 and above': 3       # High income
}

# Map the income levels to ordinal categories
cleaned_df.loc[:,'CUST_INCOME_LEVEL'] = cleaned_df['CUST_INCOME_LEVEL'].map(income_mapping)

# Display the updated DataFrame
cleaned_df
```

Fig 38: Income level into ordinal numbers

	CUST_GENDER	AGE	CUST_MARITAL_STATUS	COUNTRY_NAME	CUST_INCOME_LEVEL	EDUCATION	OCCUPATION	HOUSEHOLD_SIZE	YRS_RESIDENCE	AFFINITY_CARD	HOME_THEATER_PACKAGE	BOOKKEEI
0	0	41	NeverM	18	3	Masters	Prof.	2	4	0	1	
1	1	27	NeverM	18	3	Bach.	Sales	2	3	0	0	
2	0	20	NeverM	18	3	HS-grad	Cleric.	2	2	0	0	
3	1	45	Married	18	1	Bach.	Exec.	3	5	1	1	
4	1	34	NeverM	18	3	Masters	Sales	9+	5	1	0	
...	...	...	...	...	...	...	...	...	...	...	...	
1495	1	17	NeverM	18	1	10th	Other	1	1	0	0	
1496	1	41	Married	15	3	Bach.	Exec.	3	4	0	1	
1497	1	53	Married	18	3	HS-grad	Exec.	3	8	1	1	
1498	1	55	Married	18	1	HS-grad	Cleric.	3	7	1	1	
1499	0	40	Divorc.	18	2	HS-grad	Cleric.	2	3	0	1	

1420 rows × 14 columns

Fig 39: Results of income level transformation

EDUCATION

Fig 39 shows the Education column is mapped into ordinal number based on USA education level.

```
education_order = ['Presch.', '1st-4th', '5th-6th', '7th-8th', '9th', '10th', '11th',
                   '12th', '< Bach.', 'HS-grad', 'Assoc-A', 'Assoc-V', 'Bach.',
                   'Masters', 'Profsc', 'PhD']
cleaned_df['EDUCATION'] = pd.Categorical(cleaned_df['EDUCATION'], categories=education_order, ordered=True)
cleaned_df['EDUCATION'] = cleaned_df['EDUCATION'].cat.codes + 1
```

FIG 39: Education into ordinal numbers

```
cleaned_df
```

	CUST_GENDER	AGE	CUST_MARITAL_STATUS	COUNTRY_NAME	CUST_INCOME_LEVEL	EDUCATION	OCCUPATION	HOUSEHOLD_SIZE	YRS_RESIDENCE	AFFINITY_CARD	HOME_T
0	0	41	NeverM	18	3	14	Prof.	2	4	0	
1	1	27	NeverM	18	3	13	Sales	2	3	0	
2	0	20	NeverM	18	3	10	Cleric.	2	2	0	
3	1	45	Married	18	1	13	Exec.	3	5	1	
4	1	34	NeverM	18	3	14	Sales	9+	5	1	
...	...	...	...	...	...	...	...	...	...	...	
1495	1	17	NeverM	18	1	6	Other	1	1	0	
1496	1	41	Married	15	3	13	Exec.	3	4	0	
1497	1	53	Married	18	3	10	Exec.	3	8	1	
1498	1	55	Married	18	1	10	Cleric.	3	7	1	
1499	0	40	Divorc.	18	2	10	Cleric.	2	3	0	

1420 rows × 14 columns

FIG 40: Result of education transformation

HOUSEHOLD_SIZE

Fig 41 shows the HOUSEHOLD_SIZE column is transformed into ordinal numbers using a mapping dictionary, with each category assigned a corresponding numerical value based on the room numbers.

```
[ ] room_map = {'1':1, '2':2,'3':3,'4-5':4,'6-8':5,'9+':6}
    cleaned_df.loc[:,'HOUSEHOLD_SIZE'] = cleaned_df['HOUSEHOLD_SIZE'].map(room_map)
```

FIG 41: Household size into ordinal numbers based on room numbers

```
[ ] cleaned_df
```

	CUST_GENDER	AGE	CUST_MARITAL_STATUS	COUNTRY_NAME	CUST_INCOME_LEVEL	EDUCATION	OCCUPATION	HOUSEHOLD_SIZE	YRS_RESIDENCE	AFFINITY_CARD	HOME_THEA
0	0	41	NeverM	18	3	14	Prof.	2	4	0	
1	1	27	NeverM	18	3	13	Sales	2	3	0	
2	0	20	NeverM	18	3	10	Cleric.	2	2	0	
3	1	45	Married	18	1	13	Exec.	3	5	1	
4	1	34	NeverM	18	3	14	Sales	6	5	1	
...	...	...	...	...	...	...	...	...	...	...	
1495	1	17	NeverM	18	1	6	Other	1	1	0	
1496	1	41	Married	15	3	13	Exec.	3	4	0	
1497	1	53	Married	18	3	10	Exec.	3	8	1	
1498	1	55	Married	18	1	10	Cleric.	3	7	1	
1499	0	40	Divorc.	18	2	10	Cleric.	2	3	0	

1420 rows × 14 columns

FIG 42: Results of Household transformation

I converted other string values columns such as CUST_MARITAL_STATUS and OCCUPATION into ordinal numbers. The conversion of other variables into ordinal numbers was motivated by the necessity to facilitate machine learning model training. Since machines inherently process numerical data, converting object variables into ordinal numbers enabled a smoother analysis process.

CUST_MARITAL_STATUS

Fig 43 shows Customer marital status is converted into ordinal numbers.

```
#computing the mapping dictionary for CUST_MARITAL_STATUS
cust_marital_unique = cleaned_df['CUST_MARITAL_STATUS'].unique()
cust_marital_ordinal = {cust_marital: i for i, cust_marital in enumerate (cust_marital_unique)}

cleaned_df['CUST_MARITAL_STATUS'] = cleaned_df['CUST_MARITAL_STATUS'].apply(lambda cust_mary: cust_marital_ordinal[cust_mary]).astype(int)
cust_marital_ordinal
cleaned_df
```

FIG 43: Marital status into ordinal numbers

	CUST_GENDER	AGE	CUST_MARITAL_STATUS	COUNTRY_NAME	CUST_INCOME_LEVEL	EDUCATION	OCCUPATION	HOUSEHOLD_SIZE	YRS_RESIDENCE	AFFINITY_CARD	HOME_THEATER_PACKAGE	BOOKKEEPING_APPLICATIO
0	0	41	0	18	3	14	Prof.	2	4	0	1	1
1	1	27	0	18	3	13	Sales	2	3	0	0	1
2	0	20	0	18	3	10	Cleric.	2	2	0	0	1
3	1	45	1	18	1	13	Exec.	3	5	1	1	1
4	1	34	0	18	3	14	Sales	6	5	1	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...
1495	1	17	0	18	1	6	Other	1	1	0	0	0
1496	1	41	1	15	3	13	Exec.	3	4	0	1	1
1497	1	53	1	18	3	10	Exec.	3	8	1	1	1
1498	1	55	1	18	1	10	Cleric.	3	7	1	1	1
1499	0	40	2	18	2	10	Cleric.	2	3	0	1	1

1420 rows × 14 columns

FIG 44: Results of marital status

OCCUPATION

Fig 45 shows the code for occupation is converted into ordinal numbers.

```
#converting OCCUPATION into ordinal number
occupation_ordinal_unique = cleaned_df['OCCUPATION'].unique()
occupation_ordinal = {occupation_number : i for i, occupation_number in enumerate(occupation_ordinal_unique)}

#Applying the mapping to the occupation column
cleaned_df['OCCUPATION'] = cleaned_df['OCCUPATION'].apply(lambda occo_num: occupation_ordinal[occo_num]).astype(int)
occupation_ordinal
```

```
{'Prof.': 0,
 'Sales': 1,
 'Cleric.': 2,
 'Exec.': 3,
 'Other': 4,
 'Farming': 5,
 'Transp.': 6,
 'Machine': 7,
 'Crafts': 8,
 'Handler': 9,
 'Protec.': 10,
 'TechSup': 11,
 'House-s': 12,
 'Armed-F': 13}
```

FIG 45: Occupation into ordinal numbers

cleaned_df

	CUST_GENDER	AGE	CUST_MARITAL_STATUS	COUNTRY_NAME	CUST_INCOME_LEVEL	EDUCATION	OCCUPATION	HOUSEHOLD_SIZE	YRS_RESIDENCE	AFFINITY_CARD	HOME_THEATER_PACKAGE	BOOKKEEPING_APPLICATION
0	0	41	0	18	3	14	0	2	4	0	1	1
1	1	27	0	18	3	13	1	2	3	0	0	1
2	0	20	0	18	3	10	2	2	2	0	0	1
3	1	45	1	18	1	13	3	3	5	1	1	1
4	1	34	0	18	3	14	1	6	5	1	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...
1495	1	17	0	18	1	6	4	1	1	0	0	0
1496	1	41	1	15	3	13	3	3	4	0	1	1
1497	1	53	1	18	3	10	3	3	8	1	1	1
1498	1	55	1	18	1	10	2	3	7	1	1	1
1499	0	40	2	18	2	10	2	2	3	0	1	1

1420 rows × 14 columns

FIG 46: Results of Occupation

After converting all variables into binary or ordinal numbers, the dataset is now ready for analysis. This transformation facilitates a clearer understanding of the data and prepares it for exploration and modeling.

4. DATA ANALYSIS

Before transitioning to the modeling phase, conducting data analysis is crucial to ensure data relevance. This involves exploring various aspects of the dataset, such as calculating sums, means and standard deviations as well as assessing their relationship with other variables.

To generate summary statistics for each column, I initialized empty lists to store the mean, sum, skewness, kurtosis, and standard deviation of each variable. Then I utilized a for loop to iterate through each column in the data frame. Within the loop, I calculated the desired statistics such as mean, sum, kurtosis, and std. The computed values were then appended to their respective lists. Subsequently, I organized these metrics into a structured data frame named metrics table.

```python
# Define num_col_update
cleaned_df_data= ['CUST_GENDER','AGE','CUST_MARITAL_STATUS', 'COUNTRY_NAME', 'CUST_INCOME_LEVEL', 'HOUSEHOLD_SIZE','YRS_RESIDENCE', 'EDUCATION', 'OCCUPATION', 'AFFINITY_CARD', 'HOME_THEATER_PACKAGE', 'BOC

# Initialize lists to store computed metrics
sum_values = []
mean_values = []
std_values = []
skewness_values = []
kurtosis_values = []

# Compute metrics for each numeric column
for col in cleaned_df_data:
    col_mean = cleaned_df[col].mean()
    col_sum = cleaned_df[col].sum()
    col_std = cleaned_df[col].std()
    col_skew = cleaned_df[col].skew()
    col_kurt = cleaned_df[col].kurtosis()

    # Append computed metrics to respective lists
    sum_values.append(col_sum)
    mean_values.append(col_mean)
    std_values.append(col_std)
    skewness_values.append(col_skew)
    kurtosis_values.append(col_kurt)

# Create a DataFrame to display the computed metrics
metrics_table = pd.DataFrame({
    "Variables":cleaned_df_data,
    "Sum": sum_values,
    "Mean": mean_values,
    "Standard Deviation": std_values,
    "Skewness": skewness_values,
    "Kurtosis": kurtosis_values
})

metrics_table
```

FIG 47: Code for Summary statistics of all variables

	Variables	Sum	Mean	Standard Deviation	Skewness	Kurtosis
0	CUST_GENDER	971	0.683803	0.465155	-0.791401	-1.375623
1	AGE	54951	38.697887	13.126784	0.568577	0.079405
2	CUST_MARITAL_STATUS	1466	1.032394	1.097280	1.740270	3.528037
3	COUNTRY_NAME	23777	16.744366	4.076109	-3.242166	9.272494
4	CUST_INCOME_LEVEL	3514	2.474648	0.742397	-1.020166	-0.453534
5	HOUSEHOLD_SIZE	4142	2.916901	1.393176	0.889278	0.339478
6	YRS_RESIDENCE	5876	4.138028	1.895238	0.780784	1.721655
7	EDUCATION	14582	10.269014	2.534225	-0.339704	0.752779
8	OCCUPATION	6028	4.245070	3.226773	0.453470	-0.939797
9	AFFINITY_CARD	370	0.260563	0.439097	1.092125	-0.808403
10	HOME_THEATER_PACKAGE	820	0.577465	0.494137	-0.313978	-1.904102
11	BOOKKEEPING_APPLICATION	1251	0.880986	0.323919	-2.355670	3.554184
12	PRINTER_SUPPLIES	1420	1.000000	0.000000	0.000000	0.000000
13	Y_BOX_GAMES	399	0.280986	0.449639	0.975552	-1.049780

FIG 48: Summary statistics of all variables

5. DATA EXPLORATION

Users can see runtime histogram graphs for a chosen variable with this python program (Fig:49). Users are asked to select a variable from the dataset for which they wish to create a histogram graphic when the programs are executed. The chosen variable's histogram plot is then shown by the application. Until they decide to stop using the application, users can keep changing variables and seeing histogram graphs. This program's interactive features make it simple and quick to explore the distribution properties of the dataset. Fig 50,51 & 52 shows the out of this program. In here I tested the age & occupation and obtain the graphs

```python
import pandas as pd
import matplotlib.pyplot as plt

# Function to display histogram plot for a chosen variable
def display_histogram(variable):
    plt.figure(figsize=(8, 6))
    plt.hist(cleaned_df[variable], bins=20, color='skyblue', edgecolor='black')
    plt.title(f'Histogram of {variable}')
    plt.xlabel(variable)
    plt.ylabel('Frequency')
    plt.grid(True)
    plt.show()

# Main program loop
while True:
    print("Choose a variable to display its histogram (type 'exit' to quit):")
    print(", ".join(cleaned_df.columns))
    choice = input("Enter variable name: ")

    if choice.lower() == 'exit':
        print("Exiting the program...")
        break

    if choice in cleaned_df.columns:
        display_histogram(choice)
    else:
        print("Invalid variable name. Please try again.")
```

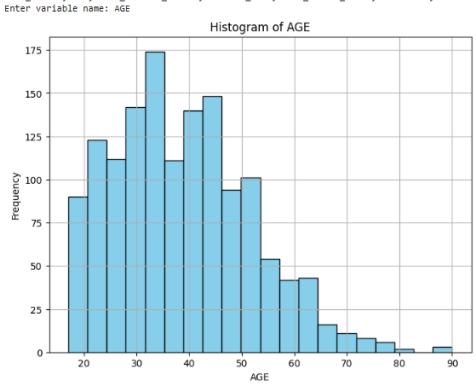
FIG 49: Code for histogram

```python
import pandas as pd
import matplotlib.pyplot as plt

# Function to display histogram plot for a chosen variable
def display_histogram(variable):
    plt.figure(figsize=(8, 6))
    plt.hist(cleaned_df[variable], bins=20, color='skyblue', edgecolor='black')
    plt.title(f'Histogram of {variable}')
    plt.xlabel(variable)
    plt.ylabel('Frequency')
    plt.grid(True)
    plt.show()

# Main program loop
while True:
    print("Choose a variable to display its histogram (type 'exit' to quit):")
    print(", ".join(cleaned_df.columns))
    choice = input("Enter variable name: ")

    if choice.lower() == 'exit':
        print("Exiting the program...")
        break

    if choice in cleaned_df.columns:
        display_histogram(choice)
    else:
        print("Invalid variable name. Please try again.")
```

```
Choose a variable to display its histogram (type 'exit' to quit):
CUST_GENDER, AGE, CUST_MARITAL_STATUS, COUNTRY_NAME, CUST_INCOME_LEVEL, EDUCATION, OCCUPATION, HOUSEHOLD_SIZE, YRS_RESIDENCE, AFFINITY_CARD, HOME_THEATER_PACKAGE, BOOKKEEPING_APPL
Enter variable name: exit
Exiting the program...
```

FIG 50: Output of histogram (1)

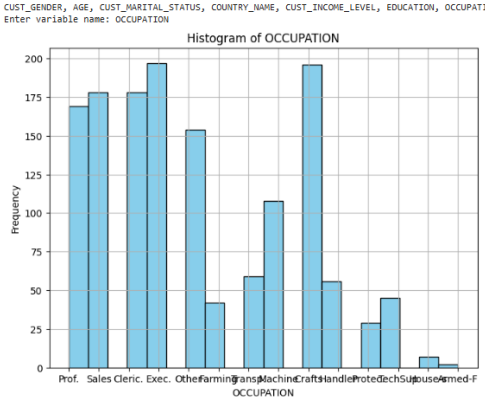
```
Choose a variable to display its histogram (type 'exit' to quit):
CUST_GENDER, AGE, CUST_MARITAL_STATUS, COUNTRY_NAME, CUST_INCOME_LEVEL, EDUCATION, OCCUPATION, HOUSEHOLD_SIZE, YRS_RESIDENCE, AFFINITY_CARD, HOME_THEATER_PACKAGE, BOOKKEEPING_APPLICATION, PRINTER_SUPPLIES, Y_BOX_GAMES
Enter variable name: AGE
```




```
Choose a variable to display its histogram (type 'exit' to quit):
CUST_GENDER, AGE, CUST_MARITAL_STATUS, COUNTRY_NAME, CUST_INCOME_LEVEL, EDUCATION, OCCUPATION, HOUSEHOLD_SIZE, YRS_RESIDENCE, AFFINITY_CARD, HOME_THEATER_PACKAGE, BOOKKEEPING_APPLICATION, PRINTER_SUPPLIES, Y_BOX_GAMES
Enter variable name:
```

FIG 51: Output of histogram (2)

```
AGE
Choose a variable to display its histogram (type 'exit' to quit):
CUST_GENDER, AGE, CUST_MARITAL_STATUS, COUNTRY_NAME, CUST_INCOME_LEVEL, EDUCATION, OCCUPATION, HOUSEHOLD_SIZE, YRS_RESIDENCE, AFFINITY_CARD, HOME_THEATER_PACKAGE, BOOKKEEPING_APPLICATION, PRINTER_SUPPLIES, Y_BOX_GAMES
Enter variable name: OCCUPATION
```




```
Choose a variable to display its histogram (type 'exit' to quit):
CUST_GENDER, AGE, CUST_MARITAL_STATUS, COUNTRY_NAME, CUST_INCOME_LEVEL, EDUCATION, OCCUPATION, HOUSEHOLD_SIZE, YRS_RESIDENCE, AFFINITY_CARD, HOME_THEATER_PACKAGE, BOOKKEEPING_APPLICATION, PRINTER_SUPPLIES, Y_BOX_GAMES
Enter variable name:
```

FIG 52: Output of histogram (3)

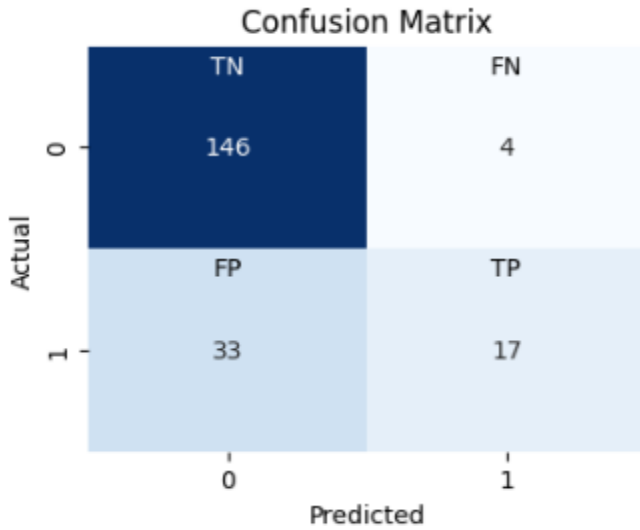
6. DATA MINING

The process of using statistical methods and computational algorithms to extract patterns, trends, and insights from big databases is known as data mining. In order to forecast future trends, find hidden patterns, and make educated decisions, it entails removing important information and expertise from raw data. Finding significant patterns and links in the data that can be utilized to address pressing issues or enhance decision making is one of the main goals of data mining.

Fig 53 illustrates the process where a subset of 1000 samples was extracted from the cleaned_df dataset and designated as random_data for subsequent logistic regression analysis. Within this random data subset, 9 out of 13 features were assigned to X while the target variable was assigned to Y, facilitation further analysis. Additionally, the construction of a confusion matrix, determination of accuracy, and plotting of the ROC curve were conducted as part of the analysis.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Select 1000 random customer records
random_data = cleaned_df.sample(n=1000, random_state=42)

# Extract features and target variable
features = ['CUST_GENDER', 'AGE', 'CUST_MARITAL_STATUS', 'COUNTRY_NAME', 'CUST_INCOME_LEVEL',
            'EDUCATION', 'OCCUPATION', 'HOUSEHOLD_SIZE', 'YRS_RESIDENCE'
            ]
target = 'AFFINITY_CARD'

X = random_data[features]
y = random_data[target]

# Split Data into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build and Train Logistic Regression Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Evaluate Model Performance
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

# Import the libraries
from sklearn import metrics
import seaborn as sns
import matplotlib.pyplot as plt

# Calculating the confusion matrix
plt.figure(figsize=(4, 3))
sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='g', cbar=False)

# Add labels and title
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')

# Add labels for TN, FN, TP, FP
plt.text(0.5, 0.1, 'TN', ha='center', va='center', color='white')
plt.text(0.5, 1.1, 'FP', ha='center', va='center')
plt.text(1.5, 0.1, 'FN', ha='center', va='center')
plt.text(1.5, 1.1, 'TP', ha='center', va='center')

# Show plot
plt.show()

print("Accuracy:", accuracy)
print("Confusion Matrix:\n", conf_matrix)
```

FIG 53: Code for Logistic Regression




```
Accuracy: 0.815
Confusion Matrix:
 [[146   4]
 [ 33  17]]
```

```
[ ] from sklearn.metrics import classification_report
    target_names = ['without Affinity card', 'with Affinity card']
    print(classification_report(y_test, y_pred, target_names = target_names))
```

```
                      precision    recall  f1-score   support

without Affinity card       0.82      0.97      0.89       150
   with Affinity card       0.81      0.34      0.48        50

             accuracy                           0.81       200
            macro avg       0.81      0.66      0.68       200
         weighted avg       0.81      0.81      0.79       200
```

FIG 54: Confusion matrix and accuracy of Logistic regression

In FIG 54, The confusion matrix is primarily used to evaluate the performance of a classification model. In here, it allows to assess the logistic regression ability to correctly classify instances into their respective classes. The logistic regression model has properly recognized 17 instances as having Affinity card. It is determined that 146 instances don't have Affinity card. 33 instances incorrectly classified as having an Affinity card. 4 instances are incorrectly classified as not having an Affinity card.

The precision for having an affinity card prediction is 0.81, meaning that 81% of customers with affinity are found to have it. The precision for customers that are expected to not have an affinity card is 0.82, meaning that 82% of customers that are expected to be without an affinity card do not have it. The recall for with affinity card is 0.34, meaning that 34% of customers who have an affinity card are properly recognized by the model. Recall for without affinity card is 0.97, meaning that 97% customers who don't have affinity card are properly recognized by the model. The harmonic mean of precision and recall yields the F1score, which offers a fair assessment of the model's effectiveness. The F1-score is 0.48 for those with an affinity card and 0.89 for customers without having affinity card. Accuracy quantifies how accurate the model's predictions are overall. In this case, the accuracy is 0.81, meaning that 81% of results are properly predicted by the model.

```
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
model.fit(X_train, y_train)
y_prob = model.predict_proba(X_test)[:,1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr,tpr)
plt.figure()
plt.plot(fpr, tpr, color ='darkorange', lw=2, label='ROC curve(area= %0.2f)' %roc_auc )
plt.plot([0,1], [0,1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

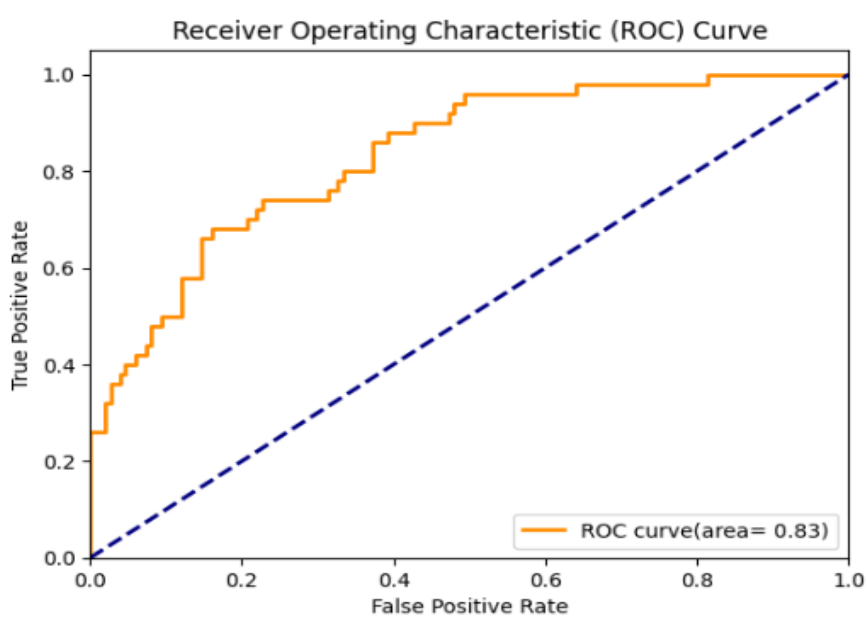


FIG 55: ROC curve for Logistic regression

ROC curves of model can assess the better performers in terms of accurately classifying positive and negative cases. The robustness of the classification model is revealed by the ROC curve's form. A curve that gets closer to the ideal point in the top left corner denotes a model with a strong capacity for discrimination.

The logistic regression appears to be successful in differentiating between cases with and without Affinity card. A curve represented by respective AUC-ROC scores of 0.83 [FIG-55]. This indicated that there is a high probability that the models will predict that customers with affinity card will have a greater anticipated likelihood than cases without, and vice versa.

Implement the prediction application

The prediction application based on the logistic regression machine learning model serves as a user-friendly tool for predicting outcomes for customer records. The program makes use of the learned logistic regression model and python to deliver precise forecasts. The program has an easy-to-use interface that allows for keyboard and file input, increasing user flexibility and convenience.

The program uses the logistic regression model to handle the data when the customer records are submitted. The model analyses various features within the customer records to predict the desired outcome, such as the likelihood of a successful marketing campaign or customer response.

The application shows the predicted answers to the user in an easy understand style once the prediction procedure is finished. Based on the application's predictions, users may quickly evaluate the data and make informed decisions. In Fig 56 shows the code for prediction application.

```python
import joblib
joblib.dump(model, 'logistic_regression_model.pkl')

import pandas as pd
from sklearn.linear_model import LogisticRegression
import joblib  # For model serialization (saving and loading)


# Load the trained logistic regression model
model = joblib.load('logistic_regression_model.pkl')

# Function to accept user input and make predictions
def predict_from_user_input():
    print("Enter customer details to predict affinity card (type 'exit' to quit)")

    while True:
        # Gather user input for customer details
        cust_gender = input("Gender (0 for female, 1 for male): ")
        if cust_gender.lower() == 'exit':
            break

        age = int(input("Age: "))
        marital_status = input("Marital Status: ")
        country_name = input("Country Name: ")
        income_level = input("Income Level: ")
        education = input("Education: ")
        occupation = input("Occupation: ")
        household_size = input("Household Size: ")
        yrs_residence = int(input("Years of Residence: "))

        # Prepare input data for prediction
        input_data = {
            'CUST_GENDER': [int(cust_gender)],
            'AGE': [age],
            'CUST_MARITAL_STATUS': [marital_status],
            'COUNTRY_NAME': [country_name],
            'CUST_INCOME_LEVEL': [income_level],
            'EDUCATION': [education],
            'OCCUPATION': [occupation],
            'HOUSEHOLD_SIZE': [household_size],
            'YRS_RESIDENCE': [yrs_residence]
        }
```

Conti.....

```
        # Create DataFrame from user input
        input_df = pd.DataFrame(input_data)

        # Make prediction
        prediction = model.predict(input_df)

        if prediction[0] == 1:
            print("Predicted: Customer will purchase affinity card.")
        else:
            print("Predicted: Customer will not purchase affinity card.")


# Function to accept file input and make batch predictions
def predict_from_file(file_path):

# Make predictions using the filtered prediction data
    features = ['CUST_GENDER', 'AGE', 'CUST_MARITAL_STATUS', 'COUNTRY_NAME', 'CUST_INCOME_LEVEL',
            'EDUCATION', 'OCCUPATION', 'HOUSEHOLD_SIZE', 'YRS_RESIDENCE'
             ]
    data_filtered = cleaned_df[features]
    predictions = model.predict(data_filtered)

    return cleaned_df

# Usage example:
predict_from_user_input()


# Example of batch prediction from a file
predicted_data = predict_from_file(cleaned_df)
print(predicted_data.head())  # Display predicted data
```

```
Enter customer details to predict affinity card (type 'exit' to quit)
Gender (0 for female, 1 for male): 0
Age: 41
Marital Status: 0
Country Name: 18
Income Level: 3
Education: 14
Occupation: 0
Household Size: 2
Years of Residence: 4
Predicted: Customer will not purchase affinity card.
Gender (0 for female, 1 for male): exit
```

FIG 56: Prediction application

```python
# Example of batch prediction from a file
predicted_data = predict_from_file(cleaned_df)
print(predicted_data.head())  # Display predicted data
```

```
Enter customer details to predict affinity card (type 'exit' to quit)
Gender (0 for female, 1 for male): 0
Age: 41
Marital Status: 0
Country Name: 18
Income Level: 3
Education: 14
Occupation: 0
Household Size: 2
Years of Residence: 4
Predicted: Customer will not purchase affinity card.
Gender (0 for female, 1 for male):
```

FIG 57: Results of prediction application (1)

In Fig 57 and 58 shows the results of prediction application, here I tested the first customer details & obtain the predictions as “will not purchase the affinity card” same as in the data.

```
Enter customer details to predict affinity card (type 'exit' to quit)
Gender (0 for female, 1 for male): 0
Age: 41
Marital Status: 0
Country Name: 18
Income Level: 3
Education: 14
Occupation: 0
Household Size: 2
Years of Residence: 4
Predicted: Customer will not purchase affinity card.
Gender (0 for female, 1 for male): exit
     CUST_GENDER  AGE  CUST_MARITAL_STATUS COUNTRY_NAME CUST_INCOME_LEVEL  \
53             1   64                   1           18                 3
309            1   27                   0            0                 3
719            1   45                   2           18                 2
1066           1   23                   0           18                 3
392            1   27                   0           18                 2

      EDUCATION  OCCUPATION HOUSEHOLD_SIZE  YRS_RESIDENCE  AFFINITY_CARD  \
53            9           1              3              6              0
309          10           4              1              3              0
719           9           8              2              7              1
1066          9           4              1              2              0
392           9           8              1              3              0

      HOME_THEATER_PACKAGE  BOOKKEEPING_APPLICATION  PRINTER_SUPPLIES  \
53                       1                        1                 1
309                      0                        1                 1
719                      1                        1                 1
1066                     0                        1                 1
392                      0                        1                 1

      Y_BOX_GAMES
53              0
309             1
719             0
1066            1
392             1
```

FIG 58: Results of prediction application (2)

Accuracy of Application

To assess the accuracy of the application, the remaining data from the **cleaned_df** dataset, comprising the remaining 420 customer details, was utilized. This dataset was assigned to variable named **remaining_data** and subsequently saved for future reference. From this remaining_data set, a sample of 100 records was randomly selected to conduct an accuracy test analysis. These 100 samples were assigned to a variable named **rest_data. (Fig 59)**

```
# Remove the selected subset from the original dataset to obtain the remaining data
remaining_data = cleaned_df.drop(random_data.index)

# Save the remaining data to a CSV file
remaining_data.to_csv('remaining_data.csv',index=False)
```

Double-click (or enter) to edit

```
[ ] import pandas as pd
    from sklearn.model_selection import train_test_split
    from sklearn.linear_model import LogisticRegressio
    from sklearn.metrics import accuracy_score
    from sklearn.preprocessing import LabelEncoder
    from sklearn.preprocessing import StandardScaler


    rest_data = remaining_data.sample(n = 100, random_state = 42)
    rest_data.head()


    X = rest_data.drop(['AFFINITY_CARD'], axis = 1)
    y = rest_data['AFFINITY_CARD']

    #Standardize the data
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # Create the logistic regression model
    model = LogisticRegression()

    # Train the model on the training data
    model.fit(X_train, y_train)

    # Make predictions on the testing data
    y_pred = model.predict(X_test)
    y_pred

    # Evaluate the model's accuracy
    accuracy = accuracy_score(y_test, y_pred)
    print("Accuracy:",accuracy)

    Accuracy: 0.7666666666666667
```

FIG 59: Code and results of Accuracy

Continuing the analysis with the rest_data set, the accuracy of the application was computed to be 0.76. This accuracy metric indicates the proportion of correctly predicted outcomes by the logistic regression model when applied to the test data. An accuracy of 0.76 suggests that the model performs reasonably well in predicting outcomes for new data samples, demonstrating its effectiveness in practical applications.

Discussion and Reflection

The coursework includes a thorough analysis of a dataset from a marketing campaign with the goal of developing prediction models and obtaining practical insights. The dataset is meticulously examined and processed through several phases of data interpretation, preparation, analysis, exploration, and mining in order to make the process of building a logistic regression machine learning model easier.

During the data understanding stage, a metadata table is created to offer details about the features and organization of the dataset. To comprehend the distribution and qualities of the data, descriptive statistics and visualizations like bar charts and histograms are used. To determine the most common values, the mode of every attribute is also calculated. Data preparation involves cleaning the dataset by addressing missing values and error data. Variables with zero influence on the target variable are identified and removed, along with error rows in the occupation column. The resulting dataset is streamlined and ready for further analysis.

Data transformation focuses on converting categorical variables into ordinal or binary representations to facilitate machine learning model training. This process involves mapping variables such as gender, country name, income level, education, marital status, and occupation into numerical formats. Data analysis involves calculating summary statistics for each column to gain insights into the dataset's distribution and characteristics. Interactive histograms graphs are one tool used in data exploration to show the distribution of variables. These graphical representations allow users to examine the attributes of the dataset and spot trends.

Building a logistic regression model with a subset of the dataset is the process of data mining. Metrics including the confusion matrix, accuracy, precision, recall, F1score, and ROC curve are used to assess the mode's performance. The logistic regression shows strong discriminating powers and a high degree of accuracy in classifying cases into the appropriate groups.

An intuitive user interface is offered by the logistic regression model-based prediction application for forecasting customer record outcomes. Based on the model's estimates, users can make educated decisions by entering client details and receiving projected responses. An accuracy metric 0.76 is obtained by evaluating the application's accuracy with a test dataset. This demonstrates the model's usefulness in practice by showing how well it predicts results for fresh data samples.

To sum up, the course offers a thorough method for dissecting and modifying a dataset from a marketing campaign, which culminates in the creation of logistic regression model for predictive analysis. Through thorough understanding, preparation, analysis, exploration and mining of the dataset, meaningful insights are extracted, and predictive models are developed to inform decision-making campaigns.

Google colab link:

https://colab.research.google.com/drive/1gYUehMB5cFmWYAuirtxv98a7K8wzchLd?usp=sharing